

Learning Planar Object Pushing with Reinforcement Learning

ME5418 Final Project Proposal

Taowen Liu, A0352191J

Xuanwei Liu, A0357433B

Chengtao Zheng, A0351467Y

Why:

- Pushing is a fundamental manipulation skill, useful when directly grasping an object is difficult or unnecessary, e.g., pre-positioning a component before grasping or assembly in industrial settings.
- Despite its simple appearance, pushing is contact-rich: the resulting object motion depends on contact location, pushing direction, object geometry, and friction, making a single hand-crafted controller hard to design for all configurations.
- In this project, we investigate whether reinforcement learning can learn a closed-loop pushing policy that moves an object from a randomized initial pose to a desired target pose, primarily for a T-shaped object similar to the Push-T task, with simpler geometries also possible.

Conventional Algorithms:

- Conventional pushing approaches rely on geometric planning, model-based control, or trajectory optimization, using an estimated object pose and contact dynamics to plan a sequence of actions toward the target. However, accurate contact dynamics are difficult to model when geometry and friction vary, so these methods often require precise physical parameters or frequent replanning.
- Instead, we believe reinforcement learning is a good fit for this problem: rather than relying on an explicit (and potentially inaccurate) contact model, an RL agent can learn a closed-loop feedback policy directly from experience, implicitly capturing how contact location and pushing direction affect the resulting motion. We favor RL specifically over supervised or imitation learning because it requires no dataset of expert demonstrations: the agent can instead discover an effective strategy on its own by directly optimizing task success through trial-and-error interaction, a natural fit for this sequential, closed-loop decision-making problem.

Problem Statement:

- We consider a tabletop planar pushing environment with a robot end-effector (pusher), a movable rigid object, and a target pose, on a bounded square tabletop (roughly $0.4\text{ m} \times 0.4\text{ m}$). The main experiment uses a T-shaped object, although simpler shapes may also be considered.
- At the start of each episode, the object's initial position and orientation are randomized within the workspace; the target pose may also be randomized.
- The robot must reach the target configuration within a limited number of control steps (e.g., 200–300 per episode), succeeding when the position error is below 2 cm and the orientation error is below 10° .
- If time permits, we will additionally randomize physical properties such as the object–table friction coefficient to test robustness under different dynamics.

RL Cast:

- **State space:** Low-dimensional robot and object states, including:
 - current pusher/end-effector position;
 - object position and orientation;

- target position and orientation;
- relative object-to-target position and orientation;
- optionally, object and robot velocities.
- **Action space:** A continuous action space controlling the planar motion of the pusher/end-effector, $a_t = [\Delta x, \Delta y]$, the desired displacement per control step. If a robotic arm is used, these commands can instead be converted into end-effector Cartesian control commands.
- **Reward structure:** We use a dense pose-error reward with normalized position and orientation terms, so their relative weights are not determined by the choice of units:

$$r_t = -w_p \frac{d_{\text{pos},t+1}}{L} - w_r \frac{d_{\text{rot},t+1}}{\pi} - c_{\text{step}} + R_s \mathbf{1}[\text{success}_{t+1}] - c_b \mathbf{1}[\text{invalid}_t].$$

Here $d_{\text{pos}} = \|p - p^*\|_2$ is measured in meters and $L = 0.4\sqrt{2}$ m is the workspace diagonal. The orientation error is the shortest angular distance, $d_{\text{rot}} = |\text{atan2}(\sin(\theta - \theta^*), \cos(\theta - \theta^*))|$, measured in radians. Pose errors are evaluated after each action, and $\mathbf{1}[\cdot]$ denotes an indicator function. Initial settings are $w_p = w_r = 1$, $c_{\text{step}} = 0.01$, $R_s = 10$, and $c_b = 1$, to be tuned on validation episodes.

Success requires both $d_{\text{pos}} < 0.02$ m and $d_{\text{rot}} < \pi/18$; the success bonus is awarded once and the episode then terminates. Episodes reaching the step limit are truncated without a success bonus. An action requesting a pusher position outside the admissible workspace is marked invalid and clipped to the boundary; this incurs c_b but does not end the episode. Workspace boundaries confine the object. We do not penalize loss of contact, since moving away and establishing a new contact may be necessary to correct orientation. The per-step cost discourages unnecessary delays. We will compare the normalized reward against the original unnormalized pose-error reward, keeping other settings fixed, to assess its effect on learning and success.

- **Neural Network Structure:** Since the observation space is low-dimensional, we plan to use a simple MLP (e.g., two hidden layers of 256 units, ReLU) for the policy and value networks, following common continuous-control PPO defaults (e.g., Stable-Baselines3), with the actor and critic sharing a trunk or fully separate if training is unstable.

RL Algorithms and Envisioned Results:

- We plan to use Proximal Policy Optimization (PPO) as the primary algorithm, given its stability and wide use in continuous robotic control; time permitting, we may additionally compare against SAC.
- We will evaluate task success rate, final position error, final orientation error, and steps-to-target, averaged over a held-out set of randomized test episodes (e.g., 100).
- The policy will be evaluated under different initial and target poses, and if time permits, under different friction coefficients or object geometries, to study generalization and robustness.
- As a baseline, we will compare against a heuristic controller that first moves the pusher to a contact point behind the object (relative to the target direction) and then pushes it toward the target, re-planning as the object-to-target direction changes. We deliberately use this lightweight heuristic rather than a full model-based planner, since the latter would itself require an accurate contact/friction model, which is exactly the dependency we are trying to avoid by using RL. We expect the learned policy to perform better on cases requiring multiple contacts or simultaneous position/orientation adjustment.